PROJECT GOLD

Documento de Design do Jogo (GDD)

Versão 0.3 — Documento consolidado de pré-produção

Status: Rascunho de trabalho consolidado

Tipo de projeto: Sandbox de terror em primeira pessoa, multijogador assimétrico e plataforma de criação de cenários

Plataforma inicial: Windows PC

Plataforma futura desejada: Linux

Engine planejada: Godot 4

Linguagens planejadas: GDScript e Lua

Arquitetura multijogador planejada: Listen server hospedado pelo Game Master

Fase atual: Pré-produção e planejamento do protótipo mínimo

Filosofia de ferramentas: Prioridade para ferramentas gratuitas e de código aberto

1. Visão geral

Project Gold é um jogo de terror multijogador assimétrico no qual um jogador assume o papel de Game Master (GM), responsável por dirigir e manipular a experiência, enquanto os demais jogadores assumem o papel de sobreviventes.

O GM não é simplesmente um espectador, uma câmera ou um administrador externo ao mundo. **Ele é um jogador pleno e uma entidade viva dentro do universo do jogo**, podendo participar diretamente dos acontecimentos e fazer parte do cânone do cenário.

O GM pode assumir uma presença física no mundo, utilizando um modelo e uma aparência escolhidos por ele, interagir com o ambiente e participar das situações vivenciadas pelos sobreviventes. Também pode se disfarçar de sobrevivente ou de

outra entidade compatível, ocultando sua verdadeira identidade conforme a experiência desejada.

Ao mesmo tempo, o GM dispõe de poderes privilegiados para dirigir a experiência: pode observar os sobreviventes, criar encontros, posicionar entidades, manipular o ambiente, executar scripts e improvisar acontecimentos em resposta às ações do grupo. Sua presença física e sua autoridade de controle são características complementares, permitindo que alterne entre participação direta, observação e direção da sessão.

Os sobreviventes exploram o mundo, interagem com objetos, utilizam armas, comunicam-se e reagem aos acontecimentos. Sua experiência pode envolver a presença do próprio GM como personagem, aliado, ameaça, figura misteriosa ou qualquer outro papel compatível com o cenário.

O jogo não exige um vencedor ou perdedor universal. Os objetivos e as condições de encerramento dependem do cenário, da campanha e das decisões do grupo.

Project Gold combinará uma experiência jogável com ferramentas de direção, edição de cenários, scripting e suporte a conteúdo personalizado. O foco principal é a experiência emergente: sustos, tensão, improvisação, exploração, situações inesperadas e diversão compartilhada.

O jogo deverá separar conceitualmente a identidade do jogador, sua representação física no mundo e suas permissões de GM. Assim, mudar de aparência, assumir um disfarce ou deixar de ser visível não deverá remover sua autoridade privilegiada, e o significado narrativo de sua presença poderá ser definido pelo cenário e por quem conduz a sessão.

2. Pilares de design

2.1. O GM é o diretor da experiência

O GM deve dispor de ferramentas poderosas para manipular o mundo e reagir aos jogadores. Sua função não é seguir obrigatoriamente um conjunto fixo de regras de vitória e derrota.

2.2. Liberdade de improvisação

O GM deve poder criar acontecimentos em tempo real, sem depender exclusivamente de eventos previamente programados.

2.3. Personalização como elemento central

Cenários, entidades, sons, scripts e outros conteúdos personalizados devem ser parte fundamental do projeto, não apenas recursos adicionais.

2.4. A experiência importa mais que a competição

O jogo não exige um vencedor ou perdedor universal. Os objetivos e as condições de encerramento dependem do cenário, da campanha e das decisões do grupo.

2.5. A complexidade deve ser opcional

O GM deve poder utilizar ferramentas simples, como menus e atalhos, ou sistemas avançados, como scripts, eventos condicionais e cenários preparados.

2.6. O jogo deve permitir experiências não previstas pelos desenvolvedores

A arquitetura deve fornecer ferramentas consistentes e extensíveis para que a comunidade crie conteúdo e mecânicas próprias.

3. Público e estrutura de uma sessão

O público-alvo inclui grupos de amigos, jogadores de terror, Game Masters, criadores de cenários e desenvolvedores de modificações.

Uma sessão típica poderá seguir este fluxo:

1. O GM seleciona um mapa, cenário preparado ou ambiente vazio.
2. Os participantes entram na sessão.
3. O grupo estabelece seus objetivos, quando necessário.
4. Os sobreviventes exploram, interagem e se comunicam.
5. O GM acompanha os jogadores e introduz acontecimentos manualmente ou por meio de sistemas automatizados.
6. A sessão termina quando o grupo, o cenário ou a campanha determina que ela terminou.

Esse fluxo é uma referência, não uma sequência obrigatória. O GM deve poder ignorar etapas, improvisar e alterar a direção da sessão.

4. Jogadores e multijogador assimétrico

Project Gold será um jogo multijogador assimétrico com dois papéis fundamentais: GM e sobreviventes.

4.1. Game Master

O GM será o anfitrião da sessão e terá acesso às ferramentas privilegiadas de controle do mundo.

Ele deverá poder:

- Manipular a própria posição e movimentação.
- Criar, posicionar, modificar e remover entidades.
- Observar os jogadores e os ambientes.
- Executar comandos e scripts.
- Controlar eventos e elementos ambientais.
- Preparar cenários antes de iniciar uma sessão.
- Improvisar durante a partida.

4.2. Sobreviventes

Os sobreviventes deverão poder:

- Movimentar-se em primeira pessoa.
- Explorar os ambientes.
- Utilizar armas e interagir com entidades e objetos compatíveis.
- Comunicar-se por voz.
- Cooperar ou agir individualmente, conforme o cenário.

Sistemas avançados de inventário, progressão, saúde, morte e combate permanecem sujeitos a decisões posteriores de design.

4.3. Quantidade de jogadores

O jogo não deverá impor um limite artificial de jogadores definido apenas por uma preferência de design.

A quantidade efetivamente suportada continuará dependendo dos limites técnicos do servidor, da rede, do desempenho e dos sistemas utilizados. O objetivo de permitir quantidades variáveis não representa uma garantia de escalabilidade ilimitada.

O criador de cenários deverá poder configurar recomendações e avisos sobre a quantidade de participantes.

Exemplos de informações exibidas:

- **Quantidade recomendada de jogadores neste cenário: 3.**
- **Este cenário poderá não funcionar corretamente com mais de 5 jogadores devido às suas mecânicas.**

A recomendação deverá ser configurável por cenário. Também será possível informar que determinadas mecânicas deixam de funcionar adequadamente acima de certa quantidade de jogadores.

Essas informações deverão orientar o grupo, não impedir artificialmente a entrada de participantes.

Para o primeiro protótipo, o alvo de teste será uma sessão com quatro conexões de jogadores no total: um GM e três sobreviventes. Essa meta inicial não constitui um limite permanente de design.

Por padrão, a contagem recomendada deverá considerar todos os participantes, incluindo o GM. O formato de apresentação poderá deixar explícita a separação entre GM e sobreviventes.

5. Ambientação e atmosfera

A ambientação inicial será predominantemente urbana e contemporânea, com ambientes familiares que possam se tornar ameaçadores por meio da iluminação, do som, da composição espacial e dos acontecimentos.

A direção de atmosfera poderá se inspirar na sensação de exploração e nas experiências emergentes de jogos cooperativos como Sven Co-op, sem copiar mapas, recursos protegidos ou conteúdo de terceiros.

5.1. Áudio

O áudio será uma ferramenta central para produzir tensão e orientar a atenção dos jogadores.

O sistema deverá contemplar, progressivamente:

- Ambientes sonoros contínuos.
- Efeitos posicionais.

- Sons distantes e falsos indícios.
- Eventos sonoros acionados pelo GM.
- Sons vinculados a objetos, locais, entidades e gatilhos.
- Conteúdo de áudio personalizado.

O GM deverá poder utilizar o áudio como parte da direção da experiência, inclusive sem revelar imediatamente a origem de um som.

6. Modos de jogo

O projeto deverá permitir diferentes formas de utilização:

Sandbox: o GM entra em um ambiente e cria a experiência livremente.

Cenário preparado: o GM carrega um cenário com eventos, entidades, gatilhos, scripts e objetivos previamente definidos.

Campanha: vários cenários podem ser conectados por progressão, transições e dados persistentes.

Cenários personalizados: criadores poderão distribuir experiências próprias, com requisitos de conteúdo, dependências e versões de API declarados.

A campanha e os sistemas avançados de progressão poderão ser desenvolvidos após a primeira experiência jogável completa.

7. Poderes e ferramentas do GM

7.1. Recursos mínimos do protótipo

O primeiro protótipo jogável deverá conter, no mínimo:

- Menu de criação de entidades.
- Atalhos de teclado para ações frequentes.
- Criação de entidades baseada na posição indicada pela mira.
- Teletransporte para a posição indicada pela mira.
- Alternância de invisibilidade.
- Alternância de noclip.
- Ferramentas básicas de movimentação e observação.
- Armas básicas para os sobreviventes.

Os controles deverão ser configuráveis quando viável, evitando depender permanentemente de teclas fixas.

7.2. Posicionamento pela mira

O GM deverá poder selecionar uma entidade no menu ou por atalho e posicioná-la utilizando a mira.

O sistema deverá buscar uma posição válida no mundo, normalmente por meio de um ponto de impacto ou mecanismo equivalente.

Posições inválidas deverão ser tratadas com segurança. O jogo deverá evitar, sempre que possível, a criação de entidades dentro de geometria sólida, fora dos limites utilizáveis ou em locais incompatíveis com a entidade.

7.3. Recursos posteriores

Após o protótipo mínimo, o sistema deverá evoluir para permitir:

- Teletransporte para jogadores, entidades e posições salvas.
- Alteração de modelos e aparências.
- Criação, remoção e modificação de entidades.
- Controle de iluminação, efeitos ambientais e áudio.
- Criação e gerenciamento de gatilhos.
- Observação por câmeras e visão livre.
- Manipulação de objetivos.
- Execução de eventos condicionais.
- Ferramentas avançadas de direção e automação.

Recursos como possessão, ilusões, transformações ambientais complexas e controle avançado de IA são possibilidades futuras, não requisitos do protótipo.

8. Comandos, menus e tratamento de erros

As ações privilegiadas do GM deverão utilizar uma camada compartilhada de comandos.

Sempre que possível, menus, atalhos, console e scripts deverão invocar a mesma operação central, com as mesmas regras de validação e permissões.

Isso evita que uma ação funcione pelo menu, mas apresente comportamento diferente quando executada por script.

8.1. Console

O console deverá exibir informações úteis sobre comandos e acontecimentos relevantes.

Comandos inválidos, conflitos, referências inexistentes e operações rejeitadas deverão gerar mensagens compreensíveis.

Exemplos:

- Entidade não encontrada.
- Posição de criação inválida.
- Operação rejeitada por conflito com outra operação.
- Script interrompido devido a um erro.
- Dependência necessária não encontrada.

8.2. Comandos simultâneos ou conflitantes

O sistema deverá detectar e tratar operações conflitantes quando possível.

Quando uma operação for rejeitada ou precisar aguardar outra, o console deverá explicar claramente o motivo e indicar o resultado da operação.

Erros esperados não deverão provocar o encerramento do jogo. O tratamento de exceções deverá preservar o estado consistente da sessão sempre que possível.

9. Controle do mundo e entidades

O sistema de entidades deverá permitir a criação e o gerenciamento de objetos do mundo, inimigos, personagens não jogáveis, itens e elementos interativos.

A remoção de entidades deverá poder ser decidida pelo GM conforme suas ferramentas e permissões. O jogo não deverá exigir que toda entidade desapareça por uma regra universal de tempo ou condição.

As regras específicas de cada entidade poderão definir condições adicionais, como dependências, estados obrigatórios ou consequências de remoção.

O estado autoritativo das entidades deverá ser controlado pelo servidor. Os clientes deverão receber as atualizações necessárias para apresentar um estado consistente do mundo.

10. Sala de controle do GM

A sala de controle deverá evoluir gradualmente para uma interface que permita dirigir e acompanhar a sessão sem obrigar o GM a navegar constantemente por menus desconectados.

A arquitetura desejada inclui:

- Interface gráfica.
- Console de comandos.
- Editor e executor de scripts.
- Ferramentas de entidades.
- Gerenciamento de cenários.
- Câmeras e vigilância.
- Histórico de eventos.
- Notificações.
- Indicadores de atividade de voz.
- Visão geral do mapa e dos jogadores.
- Ferramentas de diagnóstico.

10.1. Modularidade

Os painéis deverão ser desenvolvidos de maneira modular, permitindo reorganização e evolução independente sempre que viável.

O suporte a múltiplos monitores é um objetivo importante, mas a experiência deverá continuar funcional em um único monitor.

Janelas nativas destacáveis e configurações avançadas de múltiplos monitores poderão ser implementadas após a validação da interface básica.

11. Vigilância e notificações

O GM deverá poder observar locais relevantes sem precisar permanecer fisicamente neles.

O sistema de vigilância deverá evoluir para incluir:

- Câmeras posicionáveis.
- Identificação e nomeação de câmeras.
- Visualização de câmeras em janelas ou imagens sobrepostas.

- Múltiplos feeds.
- Notificações associadas a eventos e locais.
- Histórico de acontecimentos.
- Filtros e priorização de eventos.

Notificações poderão ser emitidas quando gatilhos forem ativados, entidades forem criadas ou removidas, objetivos mudarem ou scripts apresentarem erros.

A relação entre eventos, câmeras e locais deverá ser clara, permitindo que o GM entenda rapidamente o que ocorreu e onde ocorreu.

12. Comunicação por voz

A comunicação por voz entre sobreviventes deverá utilizar áudio posicional ou baseado em proximidade.

O GM deverá poder ouvir todos os sobreviventes independentemente da distância entre eles, de acordo com as regras de comunicação definidas para o jogo.

O suporte inicial será para Windows, com Linux como objetivo posterior.

Permanecem sujeitos a validação:

- Comportamento da voz através de paredes.
- Atenuação e oclusão.
- Reverb e efeitos espaciais.
- Comunicação global ou posicional do GM.
- Indicadores de atividade de voz.
- Compatibilidade com múltiplos clientes remotos.

Gravação e reprodução de sessões não são requisitos iniciais.

13. Criação de cenários e persistência

O jogo deverá permitir que o GM prepare cenários antes de convidar os sobreviventes.

O fluxo de criação deverá evoluir para contemplar:

- Seleção e carregamento de mapas.
- Posicionamento de entidades.
- Configuração de gatilhos.
- Definição de câmeras.

- Referências a scripts e conteúdos personalizados.
- Configuração de encontros.
- Objetivos e condições de encerramento.
- Dependências necessárias.

13.1. Separação entre cenário e sessão

O projeto deverá distinguir a configuração de um cenário do estado de uma sessão em andamento.

A definição de um cenário representa a experiência preparada pelo criador. O estado da sessão representa o que aconteceu durante uma execução específica.

Alterações temporárias da sessão não deverão modificar automaticamente o cenário original.

13.2. Salvamento como snapshot

O sistema deverá preservar o máximo possível do estado da sessão, respeitando as capacidades técnicas dos sistemas implementados.

Um snapshot poderá registrar, conforme suportado:

- Mapa e posição atual na experiência.
- Posições e estados relevantes dos jogadores.
- Entidades criadas e seus estados.
- Objetivos e progresso.
- Estado dos gatilhos.
- Temporizadores e eventos ativos.
- Estado persistente explicitamente disponibilizado pelos scripts.
- Identificadores e versões dos mods relevantes.

O sistema não deverá prometer restaurar estados arbitrários que não possam ser serializados de forma confiável. Scripts deverão utilizar mecanismos explícitos de salvamento e restauração para os dados que precisarem preservar.

13.3. Opções ao carregar

Ao carregar uma sessão salva, o jogo deverá permitir ao usuário escolher entre:

- **Carregar do ponto salvo:** restaurar o estado registrado no snapshot, respeitando as limitações e dependências existentes.

- **Recomeçar do início do nível atual:** carregar o nível desde seu estado inicial, usando os conteúdos atualmente disponíveis.
- **Cancelar:** retornar sem alterar a sessão atual.

A escolha entre restaurar o estado salvo e recomeçar o nível deverá ser apresentada de maneira clara quando houver diferenças relevantes ou quando o contexto de carregamento exigir essa decisão.

13.4. Alterações de mods

O sistema deverá identificar os mods associados ao snapshot por identificadores estáveis e registrar versões ou hashes de conteúdo quando apropriado.

Se houver alterações relevantes nos mods instalados, o usuário deverá receber um aviso antes de continuar.

Se um mod utilizado no momento do salvamento não estiver mais disponível, o jogo deverá informar que entidades, sons, modelos, scripts ou outros recursos fornecidos por ele poderão ficar indisponíveis.

O usuário deverá poder decidir entre continuar com o estado degradado, reiniciar o nível com os conteúdos atuais ou cancelar o carregamento.

O jogo deverá preservar o snapshot original e evitar sobrescrevê-lo automaticamente com um estado incompleto.

Quando tecnicamente viável, referências a conteúdo ausente deverão ser preservadas como referências não resolvidas, para que o usuário possa identificar o que precisa ser restaurado, substituído ou removido.

14. Modificações e conteúdo personalizado

A personalização será um dos fundamentos do projeto.

O objetivo é permitir que o usuário instale conteúdo compatível sem precisar reconstruir o executável do jogo.

14.1. Tipos de conteúdo planejados

O sistema deverá evoluir para aceitar:

- Áudio.
- Texturas e materiais.

- Modelos e objetos.
- Mapas.
- Entidades e inimigos.
- Itens e interações.
- Scripts.
- Gatilhos e eventos.
- Cenários e campanhas.
- Configurações e localização.

A disponibilidade efetiva de cada tipo de conteúdo dependerá da implementação e validação de seu respectivo fluxo de carregamento.

14.2. Organização e manifestos

Cada mod deverá possuir um identificador estável e um manifesto que descreva suas características.

O manifesto deverá contemplar, conforme necessário:

- Identificador único.
- Nome e descrição.
- Versão do mod.
- Versão da API exigida.
- Dependências.
- Caminhos e tipos de conteúdo.
- Autor e licença.
- Compatibilidade declarada.

O carregador deverá validar os manifestos e informar sobre arquivos ausentes, dependências incompatíveis, identificadores duplicados e versões de API não suportadas.

14.3. Compatibilidade e conflitos

O sistema deverá distinguir entre conteúdo ausente, conteúdo incompatível e conteúdo que foi atualizado desde o salvamento.

O jogo não deverá assumir que dois mods com nomes semelhantes são o mesmo mod. Identificadores estáveis deverão ser utilizados para determinar identidade e dependências.

A instalação, remoção e atualização de mods deverão ter comportamento documentado e previsível.

15. Scripting com Lua

Lua será a linguagem planejada para scripts de cenários, eventos e modificações em tempo de execução.

GDScript continuará responsável pela lógica interna principal do jogo, incluindo sistemas centrais, interface e infraestrutura de rede.

15.1. Capacidades desejadas

A API de scripting deverá permitir, progressivamente:

- Responder a eventos e gatilhos.
- Criar e modificar entidades autorizadas.
- Controlar áudio e efeitos ambientais.
- Gerenciar objetivos.
- Acionar notificações.
- Utilizar temporizadores.
- Executar sequências de acontecimentos.
- Salvar e restaurar dados explicitamente suportados.

Exemplo conceitual: quando um sobrevivente entra em um porão, um script toca um som, altera a iluminação e envia uma notificação ao GM.

Esse exemplo ilustra o comportamento desejado, não define uma API definitiva.

15.2. Segurança e estabilidade

Scripts deverão operar por meio de uma API documentada, com permissões definidas.

O sistema deverá prever:

- Relatórios de erros compreensíveis.
- Interrupção de scripts problemáticos.
- Limites de execução e consumo de recursos.
- Proteção contra ciclos e ativações repetidas.
- Tratamento de referências inválidas.
- Isolamento de falhas sempre que possível.

Conteúdo de terceiros não deverá receber acesso irrestrito ao sistema operacional por padrão.

A integração concreta entre Lua, Godot e os mecanismos de carregamento será validada em uma fase técnica posterior ao protótipo mínimo, salvo impedimentos que precisem ser investigados antes.

16. Arquitetura técnica planejada

A arquitetura inicial deverá utilizar Godot 4, GDScript, Lua e um listen server hospedado pelo GM.

O GM deverá executar o jogo e hospedar a sessão na mesma máquina.

O servidor deverá ser responsável pelo estado autoritativo do mundo, pela criação e remoção de entidades, pela validação de comandos privilegiados e pelas operações que alteram o estado compartilhado.

Os clientes deverão enviar solicitações permitidas e receber atualizações do estado do jogo. O servidor não deverá confiar cegamente em solicitações de clientes.

Os principais subsistemas previstos são:

1. Sessão e comunicação de rede.
2. Simulação do mundo e entidades.
3. Controle de permissões e comandos do GM.
4. Integração de scripts Lua.
5. Carregamento e validação de mods.
6. Cenários, snapshots e persistência.
7. Vigilância, notificações e histórico.
8. Comunicação por voz.
9. Interface modular.
10. Diagnóstico e registro de erros.

As versões exatas das ferramentas, bibliotecas de Lua e provedores de voz deverão ser fixadas após os testes apropriados.

16.1. Recuperação de desconexão do GM

Se o GM se desconectar de maneira detectável, a sessão deverá entrar em um estado seguro sempre que tecnicamente possível.

A experiência desejada é:

1. Suspender a progressão do jogo.
2. Exibir uma mensagem centralizada e claramente visível, como: **“Game Master desconectado — aguardando reconexão...”**
3. Preservar o estado recuperável da sessão.
4. Permitir a retomada quando o GM reconectar e a sessão continuar disponível.

Se o processo do GM travar ou encerrar completamente, o comportamento dependerá da capacidade de recuperação do servidor e do estado preservado.

A migração temporária do host para um sobrevivente e a posterior transferência de autoridade de volta ao GM são objetivos de investigação futura. Não deverão ser tratadas como recursos garantidos antes de serem implementadas e testadas.

O jogo deverá informar claramente quando a recuperação não for possível, evitando sugerir que o estado da sessão foi preservado integralmente quando isso não ocorreu.

17. Fases de desenvolvimento e critérios de conclusão

Fase 0 — Fundação do projeto

Objetivo: estabelecer uma base de desenvolvimento organizada.

- Criar o projeto e a estrutura do repositório.
- Definir convenções de código.
- Configurar controle de versão.
- Organizar tarefas, responsáveis e documentação.
- Garantir que um novo colaborador consiga executar o projeto.
- Estabelecer um processo de revisão de alterações.

Conclusão: o projeto pode ser obtido, executado e compilado/exportado em Windows, e a equipe possui um fluxo básico de colaboração.

Fase 1 — Protótipo mínimo multijogador

Objetivo: comprovar que o jogo assimétrico funciona.

- Implementar movimentação em primeira pessoa.
- Implementar a função básica do GM.
- Implementar armas básicas para os sobreviventes.
- Permitir que três sobreviventes se conectem ao GM.
- Sincronizar os elementos essenciais do mundo.
- Implementar criação de entidades por menu e atalhos.

- Implementar posicionamento de entidades pela mira.
- Implementar teletransporte para a mira.
- Implementar invisibilidade e noclip.
- Tratar entrada, saída e desconexão de jogadores.

Critério de conclusão: uma sessão com um GM e três sobreviventes funciona em rede, e as ações essenciais do GM produzem resultados consistentes para todos os participantes.

Fase 2 — Fundação dos comandos do GM

Objetivo: criar uma interface confiável para manipulação do mundo.

- Implementar um console básico.
- Estabelecer a camada compartilhada de comandos.
- Validar permissões e parâmetros.
- Implementar operações iniciais de criação e manipulação.
- Registrar erros e comandos rejeitados.

Critério de conclusão: ações equivalentes iniciadas por menus, atalhos e console passam pelas mesmas regras de validação e produzem resultados consistentes.

Fase 3 — Gatilhos e eventos

Objetivo: permitir que o mundo reaja às ações dos jogadores.

- Implementar gatilhos de entrada e saída.
- Implementar temporizadores.
- Permitir eventos de áudio e ambiente.
- Implementar notificações básicas.
- Registrar acontecimentos relevantes.

Critério de conclusão: um gatilho ativa um evento verificável, e o resultado pode ser observado pelos jogadores e pelo GM.

Fase 4 — Integração de Lua

Objetivo: permitir que cenários e modificações controlem acontecimentos por scripts.

- Integrar o runtime escolhido.
- Definir a primeira versão da API.
- Implementar carregamento e execução de scripts.

- Fornecer exemplos e mensagens de erro.
- Implementar mecanismos básicos de interrupção e proteção.

Critério de conclusão: um script pode responder a um gatilho e executar múltiplas ações, e um erro de script não encerra o jogo inteiro.

Fase 5 — Cenários e snapshots

Objetivo: permitir preparar, salvar e carregar experiências.

- Implementar criação e carregamento de cenários.
- Salvar entidades e gatilhos compatíveis.
- Registrar dependências.
- Separar a configuração do cenário do estado da sessão.
- Implementar snapshots do estado suportado.
- Tratar referências ausentes e erros de restauração.

Critério de conclusão: um cenário pode ser salvo e carregado, e o jogo consegue restaurar os dados suportados sem modificar silenciosamente a configuração original.

Fase 6 — Carregador de mods

Objetivo: permitir instalação de conteúdo personalizado sem reconstruir o executável.

- Implementar descoberta de mods.
- Definir e validar manifestos.
- Implementar validação de dependências.
- Suportar áudio personalizado.
- Suportar pelo menos um tipo adicional de conteúdo.
- Documentar a instalação e o diagnóstico de problemas.

Critério de conclusão: um usuário instala um mod compatível e utiliza seu conteúdo sem reconstruir o executável do jogo.

Fase 7 — Vigilância e interface modular

Objetivo: fornecer ao GM ferramentas de observação e controle.

- Implementar câmeras.
- Exibir feeds de câmera.
- Associar notificações a acontecimentos.
- Criar painéis modulares.

- Permitir utilização em um único monitor.

Critério de conclusão: o GM consegue acompanhar locais relevantes e receber notificações sem depender exclusivamente da observação direta do mundo.

Fase 8 — Comunicação por voz

Objetivo: implementar comunicação adequada à experiência.

- Implementar voz por proximidade entre sobreviventes.
- Permitir que o GM ouça os sobreviventes a qualquer distância.
- Configurar dispositivos e níveis de volume.
- Testar sessões com múltiplos clientes remotos.
- Implementar mensagens úteis para erros de configuração.

Critério de conclusão: a comunicação funciona de forma utilizável em uma sessão multijogador real, dentro dos limites do provedor escolhido.

Fase 9 — Primeiro cenário jogável completo

Objetivo: demonstrar uma experiência de terror reproduzível.

O cenário deverá incluir:

- Um ambiente urbano.
- GM e sobreviventes.
- Entidades e objetos interativos.
- Eventos de Lua.
- Áudio personalizado.
- Vigilância e notificações.
- Comunicação por voz.
- Um encerramento definido para a experiência.

Critério de conclusão: um grupo consegue jogar uma sessão curta sem intervenção de desenvolvedores, enquanto o GM improvisa acontecimentos e utiliza as ferramentas essenciais.

Fase 10 — Ferramentas avançadas de direção

- Interfaces destacáveis, se tecnicamente viáveis.
- Melhorias de múltiplas câmeras.
- Visão geral de jogadores e entidades.

- Depuração de scripts.
- Filtragem e priorização de eventos.
- Melhorias na edição e validação de cenários.

Fase 11 — Campanhas e expansão de conteúdo

- Progressão entre cenários.
- Transições e encerramentos configuráveis.
- Persistência de campanha quando suportada.
- Mais ambientes, entidades e interações.
- Melhorias na distribuição de cenários e mods.

Fase 12 — Compatibilidade e preparação para distribuição

- Empacotamento.
- Diagnóstico de falhas.
- Otimização.
- Documentação para criadores.
- Revisão de licenças e dependências.
- Testes de instalação e compatibilidade.
- Validação do suporte a Linux ou adiamento documentado.

18. Priorização de requisitos

Os requisitos serão classificados conforme sua prioridade de desenvolvimento.

P0 — Essencial para o protótipo mínimo

- Sessão assimétrica.
- GM e sobreviventes.
- Rede funcional.
- Armas básicas.
- Criação de entidades.
- Teletransporte para a mira.
- Invisibilidade e noclip.
- Tratamento básico de erros e desconexões.

P1 — Essencial para a primeira experiência utilizável

- Cenários preparados.
- Gatilhos e eventos.

- Scripting Lua.
- Snapshots e carregamento.
- Carregamento inicial de mods.
- Vigilância e notificações.
- Comunicação por voz.
- Um cenário de terror completo.

P2 — Expansão e refinamento

- Migração avançada de host.
- Interfaces nativas destacáveis.
- Sistemas avançados de campanha.
- Extensibilidade sofisticada de entidades.
- Ferramentas avançadas de depuração.
- Automação de vigilância.
- Distribuição e compatibilidade ampliadas.

A prioridade poderá ser revisada com base em testes, custos, riscos e necessidades observadas. Alterações deverão ser documentadas para que a equipe entenda por que um requisito mudou de prioridade.

19. Testes e qualidade

O projeto deverá incluir testes progressivos desde as primeiras fases.

19.1. Multijogador

Testar:

- Conexão e desconexão.
- Perda de conexão.
- Reconexão.
- Latência.
- Atualizações simultâneas.
- Solicitações inválidas.
- Criação e remoção de entidades.
- Consistência do estado compartilhado.
- Recuperação após falhas, dentro das capacidades implementadas.

19.2. Scripts

Testar:

- Erros de sintaxe.
- Referências inexistentes.
- Gatilhos repetidos.
- Sequências interrompidas.
- Consumo excessivo de recursos.
- Interrupção de scripts.
- Salvamento e restauração de dados suportados.

19.3. Conteúdo personalizado

Testar:

- Manifestos inválidos.
- IDs duplicados.
- Dependências ausentes.
- Incompatibilidade de API.
- Arquivos ausentes.
- Remoção e atualização de mods.
- Carregamento de snapshots com conteúdo indisponível.

19.4. Desempenho

Testar sessões representativas com diferentes quantidades de jogadores, entidades e eventos. Os resultados deverão orientar recomendações de jogadores e limites técnicos documentados.

20. Equipe e fluxo de trabalho

A equipe inicial é pequena, composta por desenvolvedores experientes e colaboradores em formação.

A divisão de responsabilidades deverá reduzir dependências desnecessárias:

- **Programação principal A:** rede, simulação e estado compartilhado.
- **Programação principal B:** ferramentas do GM, extensibilidade, scripts e carregamento de conteúdo.

- **Colaboradores:** tarefas delimitadas de interface, conteúdo, documentação, testes e recursos independentes.

As responsabilidades poderão mudar conforme a disponibilidade e a experiência da equipe.

Cada tarefa deverá ter um responsável, critérios de conclusão e dependências identificadas. Alterações que afetem interfaces entre sistemas deverão passar por revisão.

21. Orçamento, ferramentas e distribuição

O desenvolvimento deverá priorizar ferramentas gratuitas e de código aberto, sempre que adequadas.

O plano inicial deverá privilegiar:

- Controle de versão.
- Recursos provisórios para prototipagem.
- Testes locais antes de contratar infraestrutura.
- Conteúdo gratuito ou devidamente licenciado.
- Documentação clara de dependências.

Possíveis custos futuros incluem serviços de voz ou retransmissão, hospedagem, recursos artísticos e sonoros, ferramentas pagas e hardware.

Nenhuma dependência paga deverá ser adotada sem avaliar custo, alternativas e impacto para os jogadores.

O modelo de distribuição do jogo, o licenciamento do código e as regras de contribuição comunitária continuam como decisões futuras.

22. Riscos e mitigação

Escopo excessivo: priorizar o protótipo mínimo e dividir o desenvolvimento em fases demonstráveis.

Complexidade de rede: manter o servidor autoritativo e validar o comportamento com múltiplos clientes desde o início.

Integração de Lua: estabelecer uma API limitada e documentada, com tratamento de erros e controles de execução.

Carregamento de conteúdo: implementar tipos de mod progressivamente e validar dependências.

Persistência incompleta: definir explicitamente quais dados cada sistema pode salvar e restaurar.

Migração de host: tratar a transferência de autoridade como um problema técnico independente, sem pressupor que ela será automática.

Interfaces avançadas: garantir que as ferramentas essenciais funcionem em um único monitor antes de implementar recursos complexos de janelas.

Equipe pequena: manter tarefas delimitadas, interfaces documentadas e revisões frequentes.

Orçamento limitado: priorizar soluções gratuitas e validar custos antes de adotar serviços externos.

Compatibilidade com Linux: manter a arquitetura preparada para portabilidade, mas validar o suporte antes de prometê-lo como recurso disponível.

23. Decisões ainda abertas

Os seguintes tópicos deverão ser definidos conforme o projeto evoluir:

- Regras definitivas de saúde, morte, inventário e combate.
- Interações entre armas, inimigos e sobreviventes.
- Requisitos de rede para sessões maiores.
- Estratégia de conexão direta ou retransmissão.
- Política de reconexão e recuperação de sessões.
- Formatos finais de conteúdo personalizado.
- Mecanismos de extensão de tipos de entidade.
- Regras de compatibilidade entre versões de mods.
- Interface definitiva do editor de scripts.
- Recursos de depuração e edição durante uma sessão.
- Regras de persistência para campanhas.
- Modelo de distribuição, licenciamento e contribuições.
- Suporte final a Linux.

Essas decisões deverão ser registradas em um documento de decisões técnicas e de design, incluindo justificativas, consequências e responsáveis.

24. Validação técnica posterior ao protótipo mínimo

A proposta tecnológica não deverá ser considerada definitivamente validada apenas por constar neste documento.

Após o protótipo mínimo, a equipe deverá executar testes direcionados para:

- Integração do runtime Lua com Godot.
- Carregamento de cada tipo de conteúdo personalizado.
- Comportamento de scripts e interrupção de execuções problemáticas.
- Viabilidade do provedor de voz para comunicação por proximidade e audição global do GM.
- Limitações de janelas destacáveis e múltiplos monitores.
- Salvamento e restauração dos sistemas implementados.
- Recuperação de sessões e transferência de host.

Os testes deverão produzir conclusões objetivas: aprovado, aprovado com limitações ou não aprovado. As decisões decorrentes deverão ser registradas.

Problemas que ameacem diretamente a viabilidade do protótipo deverão ser investigados imediatamente, mesmo que pertençam a uma área cuja validação mais ampla esteja planejada para depois.

25. Definição de sucesso do primeiro protótipo

O primeiro protótipo será considerado bem-sucedido quando:

1. O GM puder hospedar uma sessão com três sobreviventes.
2. Os sobreviventes puderem se movimentar e utilizar armas básicas.
3. O GM puder criar entidades e utilizar teletransporte, invisibilidade e noclip.
4. As ações essenciais forem sincronizadas de forma consistente.
5. Comandos inválidos forem tratados com mensagens compreensíveis.
6. Desconexões não provocarem comportamentos silenciosamente incorretos.
7. A equipe puder demonstrar a experiência a outras pessoas sem precisar intervir manualmente para corrigir o funcionamento básico.

Esse marco não exige um jogo completo, um editor sofisticado, uma biblioteca extensa de mods ou uma campanha finalizada. Ele existe para comprovar a fundação do projeto.

26. Princípio orientador

Project Gold deverá fornecer ao GM os meios para criar experiências que os desenvolvedores não poderiam prever individualmente.

A função do jogo é fornecer um mundo compartilhado, ferramentas de direção, permissões, sistemas de rede e interfaces extensíveis. A função do GM é utilizar essas ferramentas para criar experiências. A comunidade deverá ampliar as possibilidades por meio de cenários, modificações e conteúdo personalizado.

O sucesso do projeto não será medido apenas pelo número de recursos implementados, mas pela capacidade prática de criar e compartilhar uma experiência de terror divertida, improvisada e memorável.